

Properties 5 & 6

Tag Storage & Matching Behavior, and Replacement Policy

CONTENTS OF THIS DOCUMENT

- **Part A — Property 5: Tag Storage and Matching.** How the DRAM cache remembers what it's holding, and how it checks whether a request is a hit or a miss.
- **Part B — Property 6: Replacement Policy.** How the DRAM cache decides what to throw out when a new line needs its spot.

PART A

1. What "Tag Storage and Matching" Means

Every cache needs to answer one basic question fast: *"Do I already have this address, or not?"* To answer that, it needs two things: a way to decide **where** to look (which slot an address belongs to), and a way to decide **whether what's sitting there is actually the right address** (comparing a stored fingerprint, called a **tag**, against the request).

Simple analogy: Picture a small parking garage with numbered spots, and a chalkboard next to each spot. Whenever a car parks, the attendant writes that car's license plate number on the chalkboard for that spot. Later, when a car pulls up asking "is my car in spot 5?", the attendant looks at spot 5's chalkboard: is anything written there at all, and if so, does it match this car's plate? Both answers have to be yes for it to count as "your car is here" (a **hit**). If the board is blank, or the plate written there belongs to a different car, that's a **miss**.

2. Why This Property Needs to Be True

This is the most basic correctness requirement any cache has. If the DRAM cache ever reported a hit for the wrong address, or failed to notice it already had the right data, every

result the simulator produces would be meaningless. This needs to be verified precisely: what exactly gets stored per slot, how the slot number is calculated, and what the actual comparison looks like.

CLAIM TO VERIFY

Each slot in the DRAM cache stores a tag (an address fingerprint) and a valid bit. A request is only ever counted as a hit when both the slot is occupied (valid) *and* its stored tag matches the tag of the incoming request.

3. Proof — Evidence From the Actual Source Code

1 What is actually stored in each slot ("the chalkboard")

inc/dram_cache_manager.h · lines 196–204

```
struct DCM_TAG_ENTRY {
    uint64_t tagDC;      // the fingerprint written on the chalkboard
    uint64_t indexDC;    // which slot this is (for bookkeeping)
    bool    validLine;   // is ANYTHING parked here right now?
    bool    dirtyLine;   // has it been written to since arriving?
    uint64_t farMemAddr; // the real address currently sitting here
};
```

There is exactly one of these records for every slot in the cache. Nothing more is stored — no separate list of "candidates," no extra bookkeeping. This is the entire chalkboard.

2 How the slot number and the tag are calculated from a real address

src/dram_cache_manager.cc · lines 202–210

```
uint64_t DRAM_CACHE_MANAGER::returnIndexDC(uint64_t address)
{
    return address % DCM_NUM_LINES; // which slot ("spot number")
}
uint64_t DRAM_CACHE_MANAGER::returnTagDC(uint64_t address)
{
    return address / DCM_NUM_LINES; // the fingerprint ("plate number")
}
```

% (remainder) picks the slot; / (whole-number division) becomes the tag. Two different addresses can land on the same slot number, but they will always get different tags — which is exactly what makes the comparison meaningful (see the worked example below).

3 The actual hit/miss comparison

src/dram_cache_manager.cc · line 569

```
e->isHit = oldValid && (e->tagDC == slot.tagDC);
```

This is the whole rule, in one line: a hit requires **both** conditions — the slot must already be occupied (`oldValid`), **and** the tag of the incoming request must equal the tag already written in that slot. If either condition fails, it is a miss.

WORKED EXAMPLE, WITH THE PROJECT'S REAL NUMBERS

This cache has `DCM_NUM_LINES = 2,097,152` slots in total (128 MiB of DRAM cache, divided into 64-byte lines). Suppose two different addresses are requested one after another:

```
address1 = 5,000,000 → index = 5,000,000 % 2,097,152 = 805,696  
→ tag = 5,000,000 / 2,097,152 = 2
```

This address is stored: slot #805,696 now has `tagDC = 2`, `validLine = true`.

```
address2 = 2,902,848 → index = 2,902,848 % 2,097,152 = 805,696  
(the SAME slot!) → tag = 2,902,848 / 2,097,152 = 1 (a DIFFERENT tag)
```

When `address2` is requested, the cache looks at slot #805,696 and finds `tagDC = 2` sitting there — but this request's own tag is 1. Since $2 \neq 1$, this is a **miss**, even though the slot is occupied. This is the exact mechanism that leads into Property 6 below: `address2` now needs to replace whatever is sitting in slot #805,696.

4. Summary of Evidence — Property 5

#	What was checked	Location	Result
1	What is stored per cache slot	inc/dram_cache_manager.h:196–204	Tag, valid bit, dirty bit, and the real address — nothing more
2	How slot number and tag are computed	src/dram_cache_manager.cc:202–210	Remainder gives the slot; whole-division gives the tag
3	The actual hit/miss decision	src/dram_cache_manager.cc:569	Hit requires valid slot AND matching tag — both, always

CONCLUSION — PROPERTY 5

The tag storage is minimal and explicit — one small record per slot — and the matching rule is a single, unambiguous boolean expression requiring both validity and an exact tag match. The worked example above shows this rule behaving correctly even in the tricky case where two different addresses land on the same slot: the tag comparison correctly identifies it as a miss rather than a false hit.

3

EVIDENCE
POINTS

2

CONDITIONS REQUIRED
FOR A HIT

1

WORKED NUMERIC
EXAMPLE VERIFIED

✓

PROPERTY
VERIFIED

1. What "Replacement Policy" Means

When a cache is full and a new line needs to go into a slot that's already occupied by *different* data, something already there has to be thrown out to make room. The rule that decides *what gets thrown out* is called the **replacement policy**. In many caches this is a real decision — for example, "throw out whichever one hasn't been used in the longest time" (called LRU). This section explains what the replacement policy is here, and why it doesn't need to be that complicated.

Simple analogy: *Picture a hotel where every guest's room number is decided by a fixed rule — say, room number = (day of the month) remainder (total rooms). There's no choice involved in picking a room for a new guest; the rule always points to exactly one specific room for any given day. Now suppose that room is already occupied by a guest from a different month. There's no committee meeting to decide who gets evicted — there is only ever one guest who could possibly be in that room, so that guest is the one who has to check out, automatically. That is exactly what happens in this DRAM cache.*

2. Why This Property Needs to Be True

The paper describes the baseline design as **direct-mapped**. A direct-mapped cache is specifically the kind of cache where each address has exactly one possible slot — not a choice between several. This is different from a "set-associative" cache, which would need an actual replacement algorithm because several different lines could compete for the same set. Verifying this property means proving two things: that each address really does map to only one possible slot, and that whatever currently occupies that slot is correctly identified and evicted (with its dirty data saved, not silently lost) when a different address needs it.

CLAIM TO VERIFY

Every address maps to exactly one possible slot (no candidate list to choose from).
When a different address needs that slot, the current occupant is identified as the "victim," its dirty data is preserved through a write-back if necessary, and it is then

unconditionally overwritten — there is no scenario where the new arrival is refused a slot it is entitled to.

3. Proof — Evidence From the Actual Source Code

1 Every address maps to exactly ONE slot — there is no list of candidates to choose from

This was already shown in Property 5, but it is the foundation of this property too:

src/dram_cache_manager.cc · line 204

```
return address % DCM_NUM_LINES;
```

A remainder operation (%) always produces exactly one output for a given address — never a set of two or more possible slots. This is what "direct-mapped" means in practice: there is nothing to pick between, because there is only ever one valid destination.

2 The current occupant is identified as the victim BEFORE anything is changed

src/dram_cache_manager.cc · lines 560–564

```
// Snapshot the OLD resident line (the potential victim) BEFORE any
// mutation
bool oldValid = slot.validLine;
bool oldDirty = slot.dirtyLine;
...
e->victimWasValid = oldValid;
e->victimWasDirty = oldDirty;
```

Before the incoming request is allowed to touch the slot at all, the code first records who was already living there and whether their data had been modified. This is the "victim" being identified — and there is only ever one possible victim, because of Evidence 1 above.

3 If the victim's data was modified, it is saved before being overwritten — nothing is silently lost

src/dram_cache_manager.cc · line 606

```

if (oldValid && oldDirty && !e->isHit) {
    e->dirtyLineAddr = oldFarAddr;
    e->handleDirtyLine = true; // triggers a write-back before eviction
}

```

A victim is only sent for a write-back if it was genuinely occupied and genuinely modified. A clean victim doesn't need this step (nothing to save — an unmodified copy already exists in the backing memory).

4 The slot is then unconditionally overwritten — the new arrival always wins

src/dram_cache_manager.cc · lines 613–620

```

slot.tagDC = e->tagDC; // the old tag is gone, replaced
slot.indexDC = e->indexDC;
slot.validLine = true;
if (!e->isWriteReq) {
    slot.dirtyLine = e->isHit ? slot.dirtyLine : false;
} else {
    slot.dirtyLine = true;
}
slot.farMemAddr = e->pkt.address;

```

This runs every single time, with no condition that could refuse the overwrite. There is no "keep the old one instead" branch anywhere in this function. Whatever was previously in the slot is gone the moment this code runs — which is exactly the "forced replacement" behaviour a direct-mapped cache is supposed to have.

4. Summary of Evidence — Property 6

#	What was checked	Location	Result
1	How many possible slots one address can map to	src/dram_cache_manager.cc:204	Exactly one (remainder operation), never a choice
2	Identification of the victim before any change	src/dram_cache_manager.cc:560–564	Old occupant's valid/dirty state snapshot first

3	Handling of a dirty victim	src/dram_cache_manager.cc:606	Write-back triggered only when genuinely needed
4	The actual overwrite of the slot	src/dram_cache_manager.cc:613–620	Unconditional — no path refuses the new arrival

CONCLUSION – PROPERTY 6

Because this is a direct-mapped cache, "replacement policy" is not an algorithm that chooses between multiple candidates — there is only ever one possible occupant for any given slot. The code correctly identifies that occupant as the victim, preserves its data with a write-back if it was dirty, and then always overwrites it for the new arrival. This matches the paper's stated baseline design exactly: a direct-mapped cache that "inserts on misses" with forced, deterministic replacement.

4

EVIDENCE
POINTS

1

POSSIBLE SLOT PER
ADDRESS (NO CHOICE)

0

DATA-LOSS PATHS
FOUND

✓

PROPERTY
VERIFIED